

Beyond Over-Refusal: Defending Indirect Prompt Injection via Latent Instruction Manifolds

Jiahao Chen¹, Rui Yin¹, Xinfeng Li², Qianli Ma³, Tianyu Du¹

Zhihui Fu⁴, Jun Wang⁴, Zhaoxiang Wang⁴, Shouling Ji¹

¹Zhejiang University, ²Hong Kong Polytechnic University,

³University of Science and Technology of China, ⁴OPPO Research Institute
{xaddwell, ruiyin, zjradty, sji}@zju.edu.cn, qianli-ma@mail.ustc.edu.cn,
{lxfmakeit, hzzhzzf, junwang.lu, steven.wangzx}@gmail.com

Abstract

Large Language Models (LLMs) have been integrated into complex ecosystems (e.g., Code Agents), while the Indirect Prompt Injection (IPI) attacks have emerged as critical barriers to their safe deployment. Attackers exploit LLMs’ indistinguishability between “instructions” and “data” to manipulate LLMs via maliciously injected instructions. Existing defenses, however, face an intractable safety-utility trade-off: most guardrails either incur high latency or suffer from severe *over-refusal*. In this paper, we first demonstrate that LLMs can separate **instruction** from **data** intrinsically with both theoretical and empirical evidence. Inspired by this insight, we propose **AEGIS** (Adaptive Ensemble Guard for Injection Shielding). **AEGIS** extracts instruction-sensitive projectors to identify malicious instructions and leverages a **Unified Multi-Layer Consensus** mechanism that aggregates topologically distinct signals across the network depth. Empirical evaluations show that **AEGIS** achieves remarkable detection performance against both heuristic and optimization-based attacks compared to baselines, highlighting its potential to mitigate IPI. Code is available at [Github](#)

1 Introduction

The development of Large Language Models (LLMs) from passive text generators to active agents has demonstrated remarkable performance across a wide range of Natural Language Processing (NLP) tasks (Taori et al., 2023; Ouyang et al., 2022). By integrating with Retrieval-Augmented Generation (RAG) and external tool APIs (e.g., search engines), LLMs now function as core components within complex application ecosystems, capable of executing code and managing user data (Shi et al., 2025; Zhang et al., 2025). However, this integration exposes a severe vulnerability within LLMs (Willison, 2022): the lack of explicit distinction between instructions (control flow) and

knowledge (external inputs) within the context.

This ambiguity enables Indirect Prompt Injection (IPI) attacks, where adversaries embed malicious commands into retrieved data streams, such as web pages or emails, to hijack the model’s behavior (Willison, 2022; Zhang et al., 2025). Such attacks can lead to severe consequences, including data exfiltration (Lin et al., 2026), unauthorized actions, and the propagation of harmful content (Hui et al., 2024; Collu et al., 2025). The demand for robust defense mechanisms has become paramount.

Existing countermeasures, however, struggle to reconcile the *Safety-Utility* trade-off. IPI Sanitization approaches, such as structured queries (Chen et al., 2025a) and token-wise filtering (Geng et al., 2025; Liu et al., 2025a), attempt to neutralize threats by restructuring or scrubbing the input stream. Since these methods necessitate the rigorous filtering of every input, they impose large utility penalties, incurring high computational overheads and often degrading the semantic integrity of complex user queries. Conversely, IPI Detection guardrails aim to identify and block malicious inputs. However, whether relying on external classifiers (Alon and Kamfonas, 2023; BlueteamAI, 2024) or internal steering mechanisms (Hung et al., 2025), these methods suffer from severe *over-refusal* (Li et al., 2025a) and compromise the model’s general availability and helpfulness.

In this work, we challenge the previous assumption that the conflation of instruction and data is indistinguishable for LLM (Zverev et al.). We argue that the key to resolving this trade-off lies in the intrinsic geometry of the LLM’s latent space (Jin et al., 2026), shaped by its two-stage training paradigm. We posit that: Pre-training forces knowledge representations to occupy a high-dimensional, high-entropy manifold to maximize information capacity, whereas **Instruction Tuning** collapses instruction representations into a separated, compact operator subspace (Park et al., 2024; Zou et al.,

2023a). Consequently, detection should not rely on black-box classification, but on identifying signatures within this compact instruction manifold.

Inspired by these insights, here we introduce **AEGIS** (Adaptive Ensemble Guard for Injection Shielding), a lightweight detection framework that secures LLMs without training. **AEGIS** distinguishes malicious instructions from benign data by filtering out superficial phrasing differences to isolate the core “imperative intent.” To further ensure robustness without over-refusal, we introduce a *Unified Multi-Layer Consensus* mechanism. Instead of relying on a single layer, which can be noisy, **AEGIS** aggregates evidence across multiple network depths. This ensures that an input is flagged as an attack only if it exhibits a persistent malicious signal throughout the model’s processing, eliminating false positives caused by transient noise. Our contributions are summarized as:

- We characterize the geometric relationship between instruction and knowledge representations, effectively distinguishing instructions from the high-entropy manifold of the untrusted external knowledge.
- We propose **AEGIS** that combines distribution-aware threshold calibration with a multi-layer voting mechanism. Beyond empirical efficacy, we derive a **theoretical error bound** for the False Positive Rate (FPR) and provide a geometric analysis for the ensemble’s robustness radius against latent perturbations.
- Extensive evaluations on eight IPI attacks with ten baselines demonstrate that **AEGIS** achieves remarkable detection performance. Crucially, it improves the safety-utility balance under standard benign workloads, reducing the FPR on benign tasks to negligible levels ($< 1\%$).

2 Related Work

2.1 Instruction Tuning

Instruction Fine-Tuning is pivotal in aligning pre-trained LLMs with human intent, transitioning them from next-token predictors to helpful assistants (Taori et al., 2023; Ouyang et al., 2022). This process typically involves supervising the model on pairs of instructions and desired outputs. While SFT enhances utility and instruction-following capabilities, recent studies suggest it fundamentally alters the geometry of the model’s latent space (Zou

et al., 2023a). Previous works have further questioned how well LLMs can intrinsically separate instructions from data (Zverev et al.). We posit that SFT collapses the representations of imperative instructions into a low-dimensional, compact operator subspace, distinct from the high-entropy manifold of general knowledge acquired during pre-training (Park et al., 2024), with more details given in Section 3.

2.2 Indirect Prompt Injection Attacks

IPI attacks exploit the intrinsic inability of LLMs to distinguish between instructions and knowledge (Willison, 2022; Chen et al., 2025d). These attacks have evolved from manual tricks to complex optimizations targeting various LLM components.

Heuristic-based Attacks. Early adversarial attempts relied on manual ingenuity and semantic patterns. Attackers employ role-playing scenarios (e.g., “DAN”), cognitive hacking, or translation wrappers to bypass safety alignment (Perez and Ribeiro, 2022; Liu et al., 2024b). Common techniques include “Context Ignoring,” where the model is commanded to disregard prior instructions, and “Fake Completion,” which mimics the start of a malicious response to induce non-compliance (Willison, 2022). While effective against naive models, these patterns are often brittle and can be mitigated through robust system prompts or supervised safety tuning.

Optimization-based Attacks. Some adversaries leverage automated optimization to generate adversarial suffixes. Gradient-based methods, such as GCG (Zou et al., 2023b) and activation-guided MCMC sampling (Li et al., 2025b), search for token sequences that maximize the probability of affirmative responses. Variations like Universal Injection (Liu et al., 2024a) aim for transferability across models. Beyond text generation, recent works utilize execution-based triggers like NeuralExec (Pasquini et al., 2024) or target specific applications, such as Tool Selection in agents (Shi et al., 2025) and GUI-based agents (Zhang et al., 2025). Additionally, Prompt Leaking strategies (PLeak) aim to extract the system prompt itself (Hui et al., 2024).

2.3 Indirect Prompt Injection Defenses

IPI Sanitization. Sanitization neutralizes threats within the untrusted input stream, including structured queries (StruQ) that enforce separation between instructions and data (Chen et al., 2025a),

and PISanitizer, which scrubs malicious patterns (Geng et al., 2025). Advanced methods like DRIP (Liu et al., 2025a) employ de-instruction training and residual fusion, while SecAlign (Chen et al., 2025c) utilizes preference optimization. Other approaches include task-specific fine-tuning (Jatmo) (Piet et al., 2024) and leveraging defensive tokens (Chen et al., 2025b). Despite their utility (An et al., 2025; Du et al., 2026), sanitization can be computationally expensive and may degrade semantic integrity. Therefore, our paper focuses on the IPI detection for evaluation and comparison.

IPI Detection. Injection detections classify inputs as benign or malicious before processing (Du et al., 2026). External guardrails, such as BERT-based classifiers (e.g., PIBert (BlutteamAI, 2024), DeBERTa (ProtectAI.com, 2023)) or Perplexity-based filters (Alon and Kamfonas, 2023), operate as separate modules but often lack context-awareness. Internal methods monitor activation patterns; for instance, Attention Tracker (Hung et al., 2025) and InjecGuard (Li and Liu, 2024) analyze internal states. Furthermore, recent works have focused on not just detecting but localizing the specific injection segments within the prompt (Jia et al., 2026, 2025). However, many existing methods still struggle with the over-refusal problem (Li et al., 2025a).

3 Methodology

3.1 Threat Model

Adversary’s Assumption. We assume the adversary has no access to the LLMs’ frozen weights θ , the secure system prompt I_{sys} , or the legitimate user query I_{user} . The adversary’s capability is strictly limited to injecting malicious textual instructions (I_{adv}) into the external data segment D_{ext} (e.g., by poisoning web pages or emails that are subsequently retrieved by the system) (Willison, 2022; Liu et al., 2024b). The adversary’s objective is to manipulate the model’s generation to bypass safety alignment, execute unauthorized commands, or perform data exfiltration (Hui et al., 2024).

Defender’s Assumption. The defender, as the model owner, possesses white-box access to the LLM during the inference phase. The defender is aware of the input segmentation boundaries (i.e., identifying the external context D_{ext}) but has no prior knowledge regarding the benign or malicious nature of that content. To be practical for real-time applications, the defense mechanism must operate under a *Low-Latency* constraint and satisfy

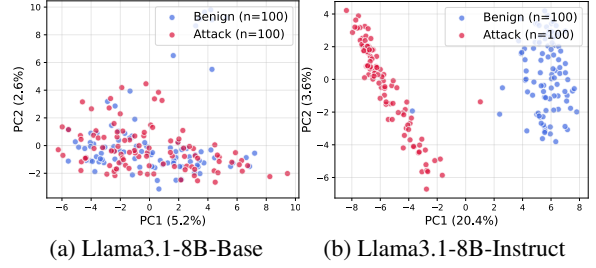


Figure 1: PCA visualization of the last-token hidden state at layer 25 on two versions of Llama3.1-8B.

a *Utility Preservation* requirement, ensuring that the suppression of malicious instructions does not impair the model’s ability to reason over benign knowledge (Li et al., 2025a).

3.2 Spectral Asymmetry and SNR

Pre-training on vast corpora forces the knowledge representations h_K to occupy a high-dimensional, high-variance manifold $\mathcal{M}_K$ to encode diverse facts. Conversely, Supervised (Instruction) Fine-Tuning (SFT) trains the model to recognize a finite set of operator patterns (e.g., “summarize”, “ignore”). Consequently, instruction representations h_I often collapse into a low-dimensional, compact subspace $\mathcal{M}_I$. As shown in Figure 1, the hidden state of the instruct model (w/ SFT) shows much higher separability than the base one (w/o SFT). Let $\Sigma_K, \Sigma_I \in \mathbb{R}^{d \times d}$ be the covariance matrices of knowledge and instructions. We posit that the spectral norms satisfy $\|\Sigma_K\|_2 \gg \|\Sigma_I\|_2$. Detecting an injection is equivalent to detecting a h_I buried in noise h_K . Even if the subspaces overlap, we can maximize the Signal-to-Noise Ratio by finding a projection direction w that minimizes the projected variance of the “signal class” (instructions) relative to the distance between class centroids via Fisher’s criterion, which motivates our design of **AEGIS**.

3.3 Latent Signal Decomposition

Consider the activation vector $h^{(l)} \in \mathbb{R}^d$ at layer l for an input sequence x . Following the Linear Representation Hypothesis (Park et al., 2024), we model $h^{(l)}$ as a superposition of semantic components. In a RAG context, a benign input consists of a knowledge component K . IPIs introduce an adversarial instruction component I_{adv} . The latent state can be formulated as:

$$h^{(l)} = h_K^{(l)} + \alpha \cdot h_{I_{adv}}^{(l)} + \epsilon \quad (1)$$

where $\alpha \in \{0, 1\}$ indicates the presence of an attack. The problem of detection is to find a projec-

tion function $f : \mathbb{R}^d \rightarrow \{0, 1\}$ that maximizes the detection probability $P(f(h) = 1 | \alpha = 1)$ while bounding the FPR $P(f(h) = 1 | \alpha = 0) \leq \tau$.

3.4 The AEGIS Framework

3.4.1 Instruction Projector

To identify IPIs, we must construct a projection operator that isolates the “instruction intent” from the background “knowledge noise.” Let $\mathcal{D}_I$ and $\mathcal{D}_K$ denote the distributions of instruction and knowledge representations, with centroids μ_I, μ_K and covariances Σ_I, Σ_K . A naïve separation (e.g., difference of means) is insufficient because instruction representations exhibit significant variance along specific “synonymy axes” (e.g., lexical variations like “Summarize” vs. “TL;DR”) while remaining invariant along the “intent axis.” We aim to find a projection direction w that maximizes the separation between the instruction signal and the knowledge, while simultaneously minimizing the variance *within* the instruction class. This ensures the detector locks onto the stable semantic core of an imperative command rather than superficial phrasing. We formulate this as:

$$w^* = \arg \max_w \frac{(w^T(\mu_I - \mu_K))^2}{w^T \Sigma_I w}, \quad (2)$$

where we use Ledoit-Wolf shrinkage to estimate Σ_I, Σ_K . Here, the numerator rewards the separation of semantic centers, while the denominator penalizes directions where instructions are unstable or noisy. Solving this optimization via Lagrange multipliers yields the closed-form **Instruction-Sensitive Projector**:

$$w^* \propto \Sigma_I^{-1}(\mu_I - \mu_K) \quad (3)$$

The term Σ_I^{-1} acts as a **Semantic Focuser** and down-weights the dimensions where instructions vary (trivial lexical features) and up-weights the dimensions where instructions are compact and distinct from knowledge. Note that we discard Σ_K in the optimization to avoid the futile compression of the knowledge manifold, focusing on collapsing the instruction manifold into a compact ‘spike’.

3.4.2 Distribution-Aware Safety Calibration

Having extracted the optimal detection direction w^* , we define a decision boundary. A fixed geometric margin is hazardous due to the heavy-tailed nature of the knowledge distribution. To avoid over-refusal without making parametric assumptions about the knowledge manifold, we introduce

a **Distribution-Aware Thresholding** mechanism. We define the decision boundary b based on the observed projection statistics of the benign knowledge set $\mathcal{D}_{benign}$. For a required safety level β (e.g., guaranteeing 99% utility retention), we set:

$$b = -\mathcal{Q}_{1-\beta}(\{w^{*T}h \mid h \in \mathcal{D}_{benign}\}) \quad (4)$$

where $\mathcal{Q}_p(\cdot)$ denotes the p -th quantile function. This mechanism ensures that the detector’s sensitivity is mathematically anchored to the model’s actual behavior on safe inputs so that the detector will trigger on at most β fraction of benign queries.

3.4.3 Multi-Layer Consensus Voting

Single-layer detectors, while calibrated, may exhibit localized sensitivity to noise or “polysemantic” activation overlaps. To capture this global consistency, we model the layers as a committee and propose a **Unified Consensus Mechanism**. Let $S_l(x) = w_l^{*T}h^{(l)}$ be the raw projection score at layer $l \in \mathcal{L}_{vote}$. We define the ensemble score $\mathcal{S}_{ens}$ as the aggregate of transformed layer-wise signals:

$$\mathcal{S}_{ens}(x) = \sum_{l \in \mathcal{L}_{vote}} \phi_l(S_l(x)) \quad (5)$$

where $\phi_l(\cdot)$ is a layer-specific transformation kernel. The final detection decision is given by $\hat{y} = \mathbb{I}(\mathcal{S}_{ens}(x) \geq \Gamma)$, where Γ is the global consensus threshold. Our framework supports two distinct instantiation modes for ϕ_l : (1) $\phi_{hard}(s) = \mathbb{I}(s > b_l)$ where b_l is the FPR-calibrated threshold and $\mathcal{S}_{ens}$ represents the raw vote count, and Γ can be the consensus quorum; (2) $\phi_{soft}(s) = (s - \mu_b^{(l)})/\sigma_b^{(l)}$ where $\mu_b^{(l)}$ and $\sigma_b^{(l)}$ are the mean and standard deviation of benign projection scores at layer l . The **visual conceptual illustration** of AEGIS framework is given in Figure 9 (for space limit).

3.5 Theoretical Error Bound of AEGIS

Consider the **Hard Voting** mode, we define the binary indicator variable $I_l = \phi_{hard}(S_l(x))$. For a benign input, we have the bound $P(I_l = 1) \leq \beta$. Assuming approximate independence (González et al., 2025) between non-adjacent layers, the ensemble FPR, denoted as $Q_{ens} = P(\mathcal{S}_{ens} \geq \Gamma)$, follows the tail of a Binomial distribution. By applying **Hoeffding’s Inequality** (Hoeffding, 1963), we derive an exponential upper bound for the system-wide error rate:

$$Q_{ens} \leq \exp \left(-2|\mathcal{L}_{vote}| \left(\frac{\Gamma}{|\mathcal{L}_{vote}|} - \beta \right)^2 \right) \quad (6)$$

This bound implies that as long as the consensus quorum ratio $\Gamma/|\mathcal{L}_{vote}|$ (e.g., 0.5 for majority rule) exceeds the single-layer error rate β (e.g., 0.1), the probability of a false alarm decays exponentially with the number of layers. This guarantee allows us to set a looser β to maintain high Recall at the layer level without compromising global safety. The analysis of soft voting is given in Appendix A.2.

3.6 Analysis of Latent Ensemble Robustness

Here, we derive a lower bound on the perturbation magnitude required to deceive the ensemble in **Hard Voting** mode. We consider a *Latent Adversary* capable of introducing an additive latent perturbation vector $\delta \in \mathbb{R}^d$ directly to the hidden state $h^{(l)}(x)$. The goal of the adversary is to force a safe input x to cross the detection boundary defined by the projector w_l^* and threshold b_l . Recall that our scoring function at layer l : $S_l(h) = \langle h, w_l^* \rangle$, and a detection is triggered if $S_l(h) \geq b_l$. Since we enforce $\|w_l^*\|_2 = 1$, the geometry of the decision boundary is a hyperplane with w_l^* .

For a benign input $h^{(l)}(x)$ correctly classified as safe (i.e., $\langle h^{(l)}(x), w_l^* \rangle < b_l$), the **Latent Robustness Radius**, denoted as $R_l(x)$, is defined as the minimum Euclidean distance from the current representation to the decision hyperplane:

$$\begin{aligned} R_l(x) &= \min_{\delta} \|\delta\|_2 \quad \text{s.t.} \quad \langle h^{(l)}(x) + \delta, w_l^* \rangle \geq b_l \\ &= \frac{b_l - \langle h^{(l)}(x), w_l^* \rangle}{\|w_l^*\|_2} = b_l - S_l(x) \end{aligned} \quad (7)$$

This equation connects our optimization to robustness: the numerator $b_l - S_l(x)$ is the margin that Eq. 2 aims to maximize. By pushing the instruction manifold away from the benign knowledge distribution, **AEGIS** enlarges this radius.

3.6.1 Theoretical Robustness of the Ensemble

We now extend to the multi-layer ensemble. An adversary must successfully flip the decision of at least Γ layers to trigger a system-wide false positive, a simultaneous constraint on the perturbation.

Proposition 1 (Ensemble Latent Robustness Radius). *Let $\mathcal{R}(x) = \{R_1(x), R_2(x), \dots, R_{|\mathcal{L}|}(x)\}$ be the set of exact latent radii for all monitored layers calculated via Eq. (7). Let $R_{(\Gamma)}$ denote the Γ -th smallest value in $\mathcal{R}(x)$ (i.e., the Γ -th order statistic). The latent robustness radius of the **AEGIS** ensemble, $R_{ens}(x)$, is exactly determined by:*

$$R_{ens}(x) = R_{(\Gamma)} \quad (8)$$

Any latent perturbation δ with $\|\delta\|_2 < R_{(\Gamma)}$ is mathematically guaranteed to preserve the safe classification of the ensemble.

Geometric Interpretation. The proposition above formalizes that the ensemble is as strong as its “median” link, rather than its weakest. Even if a specific layer k has a small margin $R_k \approx 0$, the attacker must simultaneously satisfy the geometric constraints of Γ in different semantic spaces. Finding a single vector δ that is collinear with Γ , different normals is geometrically constrained, resulting in a larger effective radius $R_{(\Gamma)}$.

4 Experiments

4.1 Experimental Settings

Evaluation Datasets. Following previous works (Jia et al., 2025; Hung et al., 2025), we utilize the *OpenPromptInjection* benchmark (Liu et al., 2024b) for evaluation with 7 NLP tasks: duplicate sentence detection, grammar correction, hate speech detection, natural language inference, sentiment analysis, spam detection, and text summarization, with 100 instruction-data pairs per task. **IPIs.** We evaluate **AEGIS** against a diverse spectrum of IPIs: (1) *Heuristic-based attacks*: following the benchmark (Liu et al., 2024b), we include Naive Attack (Harang, 2023), Escape Characters (Perez and Ribeiro, 2022), Context Ignoring (Willison, 2022), Fake Completion (Willison, 2022), and Combined Attack (Liu et al., 2024b). (2) *Optimization-based attacks*: we employ Universal Injection (Liu et al., 2024a), NeuralExec (Pasquini et al., 2024), and PLeak (Hui et al., 2024). All attacks utilize standard parameter configurations from their respective open-source implementations. Each attack was implemented on the datasets provided above (injected with the dataset samples above), consisting of $7 \times 7 \times 100 \times 8$ attack samples.

Evaluation Metrics. Following previous works (Li et al., 2025a; Liu et al., 2024b), we use accuracy (ACC), false positive rate (FPR), false negative rate (FNR), true positive rate (TPR), F1 score, and computation overhead (time, GPU memory) as metrics. Note that FNR measures the fraction of the malicious samples that are falsely detected as clean.

Defense Implementations. We leverage the a balanced training corpus (Zou et al., 2025), consisting of 10,000 benign samples from the processed Alpaca (Taori et al., 2023) and Natural Questions (Kwiatkowski et al., 2019) with 10,000 negative samples from the Naive Attack (Liu et al.,

Attack ↓	Defense →	AttTracker	DeBERTa	InjecGuard	KA	LLM Naive	PIBert	PPL ¹	PPL ²	PrGuard	PIShield	AEGIS
Combined Attack	ACC ↑	0.8071	0.6219	0.9836	0.7195	0.6710	0.6050	0.4508	0.5182	0.5371	0.5758	0.9848
	FPR ↓	0.3857	0.0057	0.0243	0.4931	0.6069	0.7900	0.5492	0.7100	0.9257	0.8171	0.0290
	FNR ↓	0.0000	0.7504	0.0086	0.0680	0.0510	0.0000	0.5492	0.2537	0.0000	0.0312	0.0014
Escape Character	ACC ↑	0.8071	0.4996	0.6005	0.6701	0.5560	0.6050	0.3996	0.4597	0.5371	0.5790	0.9852
	FPR ↓	0.3857	0.0057	0.0243	0.4967	0.6029	0.7900	0.6004	0.7100	0.9257	0.8171	0.0290
	FNR ↓	0.0000	0.9951	0.7747	0.1631	0.2851	0.0000	0.6004	0.3706	0.0000	0.0249	0.0006
Fake Completion	ACC ↑	0.8071	0.4999	0.6124	0.7082	0.6254	0.6050	0.4347	0.5051	0.5371	0.5769	0.9848
	FPR ↓	0.3857	0.0057	0.0243	0.4963	0.6024	0.7900	0.5653	0.7100	0.9257	0.8171	0.0290
	FNR ↓	0.0000	0.9945	0.7508	0.0873	0.1467	0.0000	0.5653	0.2798	0.0000	0.0290	0.0013
Context Ignoring	ACC ↑	0.8071	0.6873	0.9823	0.7056	0.4616	0.6050	0.4535	0.4990	0.5371	0.5755	0.9850
	FPR ↓	0.3857	0.0057	0.0243	0.4971	0.6155	0.7900	0.5465	0.7100	0.9257	0.8171	0.0290
	FNR ↓	0.0000	0.6196	0.0110	0.0916	0.4612	0.0000	0.5465	0.2920	0.0000	0.0318	0.0021
Naive Attack	ACC ↑	0.8071	0.4996	0.6005	0.7018	0.4585	0.6050	0.4231	0.4752	0.5371	0.5783	0.9850
	FPR ↓	0.3857	0.0057	0.0243	0.4994	0.6051	0.7900	0.5769	0.7100	0.9257	0.8171	0.0290
	FNR ↓	0.0000	0.9951	0.7747	0.0969	0.4780	0.0000	0.5769	0.3396	0.0000	0.0263	0.0010
NeuralExec	ACC ↑	0.8071	0.5620	0.9879	0.7404	0.6964	0.6050	0.7867	0.6413	0.5371	0.5786	0.9847
	FPR ↓	0.3857	0.0057	0.0243	0.4957	0.6020	0.7900	0.2133	0.7100	0.9257	0.8171	0.029
	FNR ↓	0.0000	0.8702	0.0000	0.0235	0.0051	0.0000	0.2133	0.0073	0.0000	0.0257	0.0016
PLeak	ACC ↑	0.8071	0.8170	0.8429	0.6180	0.6391	0.6050	0.6271	0.6106	0.5371	0.5893	0.9855
	FPR ↓	0.3857	0.0057	0.0243	0.4945	0.6035	0.7900	0.3729	0.7100	0.9257	0.8171	0.0290
	FNR ↓	0.0000	0.3602	0.2900	0.2696	0.1184	0.0000	0.3729	0.0688	0.0000	0.0043	0.0000
Universal Injection	ACC ↑	0.8071	0.5304	0.9879	0.7371	0.6118	0.6050	0.5963	0.6029	0.5371	0.5774	0.9850
	FPR ↓	0.3857	0.0057	0.0243	0.4922	0.6035	0.7900	0.4037	0.7100	0.9257	0.8171	0.0290
	FNR ↓	0.0000	0.9335	0.0000	0.0335	0.1729	0.0000	0.4037	0.0843	0.0000	0.0280	0.0010

Table 1: Overall detection performance (ACC, FPR and FNR) comparison between AEGIS and baselines.

2024b) and NeuralExec (Pasquini et al., 2024). Specifically, we only select 200 samples randomly from the corpus for w^* estimation. For evaluated models, we include three mainstream models including Qwen3-4B (default in evaluation unless specified) (Team, 2025), Llama3-8B (AI@Meta, 2024) and Llama3.1-8B (Grattafiori et al., 2024). More details are given in Appendix A.4.

Baselines. We compare our method with both industrial and research detections, including (1) PIBert (BlutteamAI, 2024), ProtectAI-deberta (DeBERTa) (ProtectAI.com, 2023), and PromptGuard (PrGuard) (Meta, 2024); (2) PPL (Alon and Kamfonas, 2023) (GPT2 for PPL¹ and Vicuna-7B for PPL²), Known-Answer (KA) (Nakajima, 2022), InjecGuard (Li et al., 2025a; Li and Liu, 2024), AttentionTracker (AttTracker) (Hung et al., 2025), LLM-Naive (Armstrong, 2023) and PIShield (Zou et al., 2025). The details and implementation of these baselines can be found in Appendix A.5.

4.2 Main Results

Performance Comparison. Table 1 presents the comprehensive evaluation results across 8 IPIs. We observe that **AEGIS** consistently outperforms baseline methods, achieving SOTA ACC while maintaining a negligible FPR. Note that FPR of all detection against different attacks is almost the same since the benign samples to calculate FPR

are the same for different attacks. The primary limitation of existing defenses is the imbalance between detection sensitivity and benign utility preservation. As shown in Table 1, baseline methods typically gravitate towards one extreme: (1) *Over-Refusal*: Methods like AttTracker and PrGuard achieve low FNR but suffer from high FPRs (38.57% and 92.57%, respectively), indicating a large portion of benign queries are incorrectly rejected. (2) *Under-Detection*: Conversely, shallow classifiers such as DeBERTa and PIBert maintain low FPRs ($\approx 0.5\%$) but fail to generalize to heuristic attacks, exhibiting FNRs as high as 99.51% on Naive Attacks and 99.45% on Fake Completion. In contrast, **AEGIS** achieves an average ACC exceeding 98% across most tasks. Crucially, it maintains a consistent FPR of **0.29%**. This confirms that **AEGIS** can identify malicious intent without compromising the model’s ability to process safe queries. The score distribution of **AEGIS** on different datasets and attacks be found in Figure 10.

Also, **AEGIS** demonstrates superior generalization capabilities against both attack paradigms. It effectively detects *heuristic-based attacks*, where static embeddings often fail due to lexical variations. Furthermore, it remains robust against sophisticated *optimization-based attacks*, achieving $> 98\%$ accuracy where perplexity-based methods

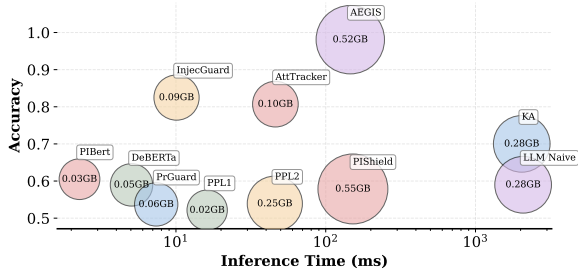


Figure 2: Efficiency-accuracy trade-off comparison.

struggle (e.g., PPL¹ achieves only 59.63% accuracy on Universal Injection). This validates the effectiveness of the multi-layer consensus mechanism in capturing deep semantic anomalies that persist across the network depth.

Overhead Comparison. Results from Qwen3-4B in Figure 2 visualizes the trade-off between detection accuracy and inference latency, where bubble sizes correspond to peak GPU memory consumption for each samples (average token length ≥ 100). We observe that much external guardrails such as PIBert and DeBERTa achieve minimal latency (< 10 ms) and memory usage (< 0.1 GB). However, their low accuracy ($\approx 60\%$) renders them insufficient for defending against sophisticated injection attacks. However, LLM-based methods like LLM Naive achieve moderate accuracy but incur prohibitive latency overheads (> 1000 ms). **AEGIS** occupies the optimal position on the Pareto frontier among baselines. While it introduces a marginal latency increase (≈ 120 ms) compared to static classifiers, it achieves near-perfect detection accuracy ($> 98\%$). Furthermore, **AEGIS** remains computationally efficient compared to the substantial resource demands of distinct LLM-based defenses, demonstrating a favorable balance between robust safety and deployment feasibility.

4.3 Ablation Studies

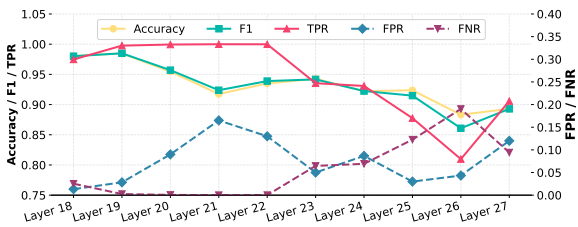


Figure 3: Detection performance of different layers.

Performance Across Layers. A core premise of our work is that instruction and knowledge representations exhibit distinct spectral signatures at spe-

cific depths of the network. Results from Qwen3-4B in Figure 3 illustrate detection performance across layers 18 to 27. We observe that detection accuracy peaks and FPR is minimized in the middle layers, where the semantic separation between instructions and knowledge is most pronounced. While in deeper layers, the FNR increases significantly. This suggests that as the model approaches the output stage, the instruction manifold collapses into the token generation subspace, making the malicious intent harder to distinguish from benign generation. This finding justifies our strategy of targeting intermediate layers for projector extraction.

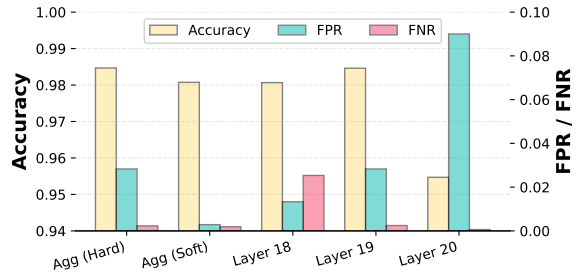


Figure 4: Ablation study of detection settings.

Impact of Multi-Layer Consensus. We evaluate the efficacy of the Unified Multi-Layer Consensus mechanism by comparing it against single-layer detectors. As shown in Figure 4, result from Qwen3-4B, while individual layers (e.g., Layer 19) can achieve high accuracy, they often exhibit instability in either FPR or FNR. The ensemble approaches, both *Agg (Hard)* and *Agg (Soft)*, effectively mitigate this variance. Specifically, the ensemble mechanisms maintain high accuracy ($> 97\%$) while stabilizing the FPR below individual layer fluctuations. This confirms that aggregating topologically distinct signals filters out layer-specific noise, creating a robust detection boundary.

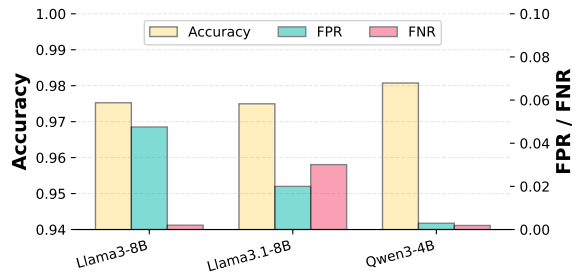


Figure 5: Detection performance of different models.

Cross-Model Generalization. To verify that **AEGIS** captures fundamental properties of LLMs rather than architecture-specific artifacts, we

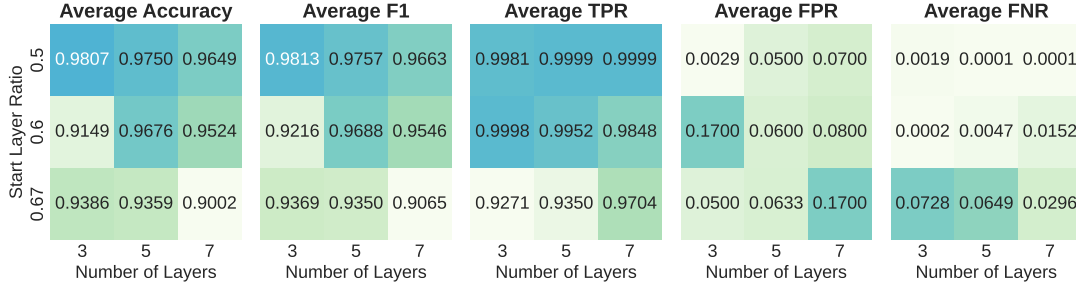


Figure 6: Detection performance (Qwen3-4B) with different numbers and positions of aggregated layers.

evaluate its performance on other LLM families. Figure 5 demonstrates the detection metrics on Llama-3-8B, Llama-3.1-8B, and Qwen3-4B. **AEGIS** achieves consistent high performance (Accuracy > 97%, FPR < 5%) across all tested models. This universality suggests that the spectral asymmetry between instruction and knowledge is a generic structural invariant introduced by the alignment process, rendering **AEGIS** applicable to a wide range of LLMs (More details in Table 2).

Sensitivity to Aggregation Hyperparameters.

Figure 6 presents a heatmap analysis of the ensemble performance under varying start layer ratios (position in the network) and the number of voting layers ($|\mathcal{L}_{vote}|$). Starting aggregation at the midpoint (0.5) of the network yields the highest Average Accuracy and TPR. Shifting the window to later layers (Start Ratio 0.67) leads to a degradation in performance, characterized by increased FNR (lighter regions in the bottom-right heatmap). Furthermore, increasing the number of layers from 3 to 7 improves stability up to a saturation point.

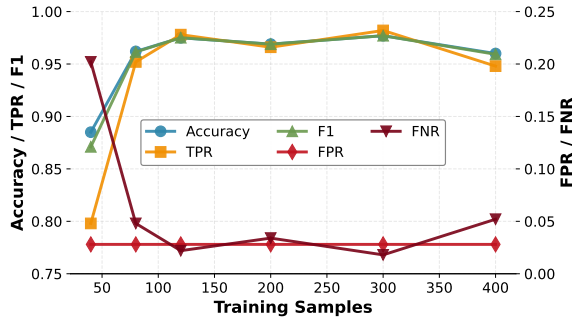


Figure 7: Detection performance (Llama3.1-8B) with different scales of training samples.

Data Efficiency. A practical defense must not require extensive datasets for calibration. We investigate the data efficiency of **AEGIS** by varying the number of overall training samples (benign/malicious pairs) used to estimate the instruction-sensitive projector. Figure 7 and Fig-

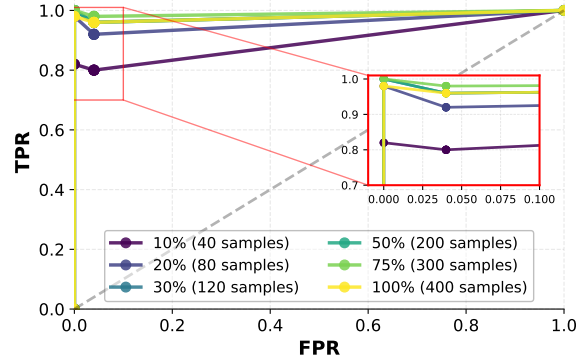


Figure 8: ROC curves (Llama3.1-8B) with different scales of training samples.

ure 8 illustrate the performance and ROC curves, respectively, as the training sample size increases from 40 (20+20) to 400 (200+200). Remarkably, **AEGIS** converges to optimal performance with as few as 100 samples. The accuracy and F1 scores plateau quickly, and the ROC curves in Figure 8 show that the area under the curve (AUC) remains high even in low-data regimes. This high data efficiency proves that the instruction manifold is low-dimensional and stable, requiring minimal samples to characterize its principal components.

5 Conclusion

In this work, we analyze the conflation of instructions and data renders LLMs susceptible to PI. Our theoretical analysis identified the spectral asymmetry between the compact instruction manifold and the knowledge manifold as a critical leverage point. Based on this, we proposed **AEGIS**, enabling the extraction of instruction-sensitive projectors. Furthermore, we formalized the robustness of our approach through a Unified Multi-Layer Consensus mechanism with layer-wise calibration and ensemble voting. Empirical evaluations across diverse benchmarks demonstrate that **AEGIS** effectively detects indirect injections, achieving SOTA detection rates with negligible impact on benign utility.

Limitations

While **AEGIS** offers robust defense guarantees, we acknowledge several limitations that define the scope of its applicability: (1) **AEGIS** requires access to the internal hidden states (activations) of the protected model, limiting its direct applicability to closed-source commercial models; (2) the error bound of **AEGIS** is based on the assumption of layer independence, which might not always hold (González et al., 2025), leading to Equation 6 be an optimistic upper bound; (3) **AEGIS** introduces a non-zero computational overhead during the detection, which may impact latency in ultra-high-throughput scenarios. These limitations can also be found in other IPI studies (Liu et al., 2025b; Jia et al., 2026; Chen et al., 2025a; Geng et al., 2025; Piet et al., 2024; Chen et al., 2025e), regardless of their strategies.

Ethical Considerations

We strictly adhere to the ACM Code of Ethics and the ACL Code of Conduct throughout the research. The primary objective of this study is to enhance the security and robustness of LLMs by proposing **AEGIS** to mitigate the risks of IPI in real-world applications. Our experiments rely exclusively on established public benchmarks. All data used in this study is anonymized and contains no personally identifiable information. We do not introduce or release any new datasets containing harmful, toxic, or offensive content beyond what is already available in the public domain for research purposes. Our commitment to transparency is reflected in making **AEGIS** fully open-source upon publication, fostering collaboration, and allowing others to verify, replicate, and build upon our work for the betterment of the field.

While our work focuses on enhancing defensive mechanisms against prompt injection attacks, we acknowledge the potential for dual use in security research. We encourage the ethical and responsible use of **AEGIS** to improve LLM security and not for malicious purposes.

LLM Usage Considerations

We utilized Large Language Models (specifically Gemini3-Pro and GPT-5) to assist with checking grammatical errors, refining sentence structure, and polishing the clarity of the text. All scientific claims, experimental designs, theoretical deriva-

tions, and the final decision on the content remain the sole responsibility of the human authors.

References

- AI@Meta. 2024. [Llama 3 model card](#).
- Gabriel Alon and Michael Kamfonas. 2023. Detecting language model attacks with perplexity. *arXiv preprint arXiv:2308.14132*.
- Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. 2025. Ipi-guard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1023–1039.
- R Gorman Stuart Armstrong. 2023. [Using gpt-eliezer against chatgpt jailbreaking](#).
- BlueteamAI. 2024. [fmops/distilbert-prompt-injection](#).
- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025a. {StruQ}: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400.
- Sizhe Chen, Yizhu Wang, Nicholas Carlini, Chawin Sitawarin, and David Wagner. 2025b. Defending against prompt injection with a few defensivetokens. *arXiv preprint arXiv:2507.07974*.
- Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025c. Secalign: Defending against prompt injection with preference optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 2833–2847.
- Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, and Bryan Hooi. 2025d. [Can indirect prompt injection attacks be detected and removed?](#) In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18189–18206, Vienna, Austria. Association for Computational Linguistics.
- Yulin Chen, Haoran Li, Zihao Zheng, Dekai Wu, Yangqiu Song, and Bryan Hooi. 2025e. [Defense against prompt injection attack by leveraging attack techniques](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18331–18347, Vienna, Austria. Association for Computational Linguistics.
- Matteo Gioele Collu, Umberto Salviati, Roberto Confalonieri, Mauro Conti, and Giovanni Apruzzese. 2025. Publish to perish: Prompt injection attacks on llm-assisted peer review. *arXiv preprint arXiv:2508.20863*.

- Mengyao Du, Han Fang, Haokai Ma, Jiahao Chen, Kai Xu, Qunjun Yin, and Ee-Chien Chang. 2026. Snapguard: Lightweight prompt injection detection for screenshot-based web agents. *arXiv preprint arXiv:2604.25562*.
- Runpeng Geng, Yanting Wang, Chenlong Yin, Minhao Cheng, Ying Chen, and Jinyuan Jia. 2025. Pisanitizer: Preventing prompt injection to long-context llms via prompt sanitization. *arXiv preprint arXiv:2511.10720*.
- Ramón Calvo González, Daniele Paliotta, Matteo Pagliardini, Martin Jaggi, and François Fleuret. 2025. [Leveraging the true depth of llms](#). *CoRR*, abs/2502.02790.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Rich Harang. 2023. Securing llm systems against prompt injection. *Online*, <https://developer.nvidia.com/blog/securing-llm-systems-against-prompt-injection>.
- Wassily Hoeffding. 1963. Probability inequalities for sums of bounded random variables. *Journal of the American statistical association*, 58(301):13–30.
- Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3600–3614.
- Kuo-Han Hung, Ching-Yun Ko, Amrith Rawat, I-Hsin Chung, Winston H Hsu, and Pin-Yu Chen. 2025. Attention tracker: Detecting prompt injection attacks in llms. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2309–2322.
- Yuqi Jia, Yupei Liu, Zedian Shao, Jinyuan Jia, and Neil Gong. 2025. Promptlocate: Localizing prompt injection attacks. *arXiv preprint arXiv:2510.12252*.
- Yuqi Jia, Yupei Liu, Zedian Shao, Jinyuan Jia, and Neil Zhenqiang Gong. 2026. Promptlocate: Localizing prompt injection attacks. In *IEEE Symposium on Security and Privacy*.
- Zehao Jin, Ruixuan Deng, Junran Wang, Xinjie Shen, and Chao Zhang. 2026. [Beyond steering vector: Flow-based activation steering for inference-time intervention](#).
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, and 1 others. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.
- Hao Li and Xiaogeng Liu. 2024. Injecguard: Benchmarking and mitigating over-defense in prompt injection guardrail models. *arXiv preprint arXiv:2410.22770*.
- Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao. 2025a. [PIGuard: Prompt injection guardrail via mitigating overdefense for free](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30420–30437, Vienna, Austria. Association for Computational Linguistics.
- Minghui Li, Hao Zhang, Yechao Zhang, Wei Wan, Shengshan Hu, Pei Xiaobing, and Jing Wang. 2025b. [Transferable direct prompt injection via activation-guided MCMC sampling](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1966–1978, Suzhou, China. Association for Computational Linguistics.
- Ruixiao Lin, Qingming Li, Jiahao Chen, Chunyi Zhou, and Shouling Ji. 2026. Sope: Situation-aware and statistically indistinguishable privacy exfiltration for mcp-enabled agents. In *Forty-third International Conference on Machine Learning*.
- Ruofan Liu, Yun Lin, and Jin Song Dong. 2025a. Drip: Defending prompt injection via de-instruction training and residual fusion model architecture. *arXiv preprint arXiv:2511.00447*.
- Xiaogeng Liu, Zhiyuan Yu, Yizhe Zhang, Ning Zhang, and Chaowei Xiao. 2024a. Automatic and universal prompt injection attacks against large language models. *arXiv preprint arXiv:2403.04957*.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024b. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*.
- Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. 2025b. Datasentinel: A game-theoretic detection of prompt injection attacks. In *IEEE Symposium on Security and Privacy*.
- Meta. 2024. [meta-llama/prompt-guard-86m](#).
- Yohei Nakajima. 2022. [Yohei’s blog post](#).
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.
- Kiho Park, Yo Joong Choe, and Victor Veitch. 2024. The linear representation hypothesis and the geometry of large language models. In *International Conference on Machine Learning*, pages 39643–39666. PMLR.

Dario Pasquini, Martin Strohmeier, and Carmela Troncoso. 2024. Neural exec: Learning (and learning from) execution triggers for prompt injection attacks. In *Proceedings of the 2024 Workshop on Artificial Intelligence and Security*, pages 89–100.

Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*.

Julien Piet, Maha Alrashed, Chawin Sitawarin, Sizhe Chen, Zeming Wei, Elizabeth Sun, Basel Alomair, and David Wagner. 2024. Jatmo: Prompt injection defense by task-specific finetuning. In *European Symposium on Research in Computer Security*, pages 105–124. Springer.

ProtectAI.com. 2023. [Fine-tuned deberta-v3 for prompt injection detection](#).

Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. 2025. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Stanford alpaca: An instruction-following llama model.

Qwen Team. 2025. [Qwen3 technical report](#). *Preprint*, arXiv:2505.09388.

Simon Willison. 2022. Prompt injection attacks against gpt-3. *Simon Willison’s Weblog*.

Yang Zhang, Yanfei Dong, and Kenji Kawaguchi. 2024. Investigating layer importance in large language models. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 469–479.

Yitong Zhang, Ximo Li, Liyi Cai, and Jia Li. 2025. Realistic environmental injection attacks on gui agents. *arXiv preprint arXiv:2509.11250*.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xu Wang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, and 1 others. 2023a. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023b. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.

Wei Zou, Yupei Liu, Yanting Wang, Ying Chen, Neil Gong, and Jinyuan Jia. 2025. Pishield: Detecting prompt injection attacks via intrinsic llm features. *arXiv preprint arXiv:2510.14005*.

Egor Zverev, Sahar Abdelnabi, Mario Fritz, and Christoph H Lampert. Can llms separate instructions from data? and what do we even mean by that? In *ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*.

A Appendix

A.1 Derivation of the Ensemble Error

Here, we provide the derivation of the exponential error bound for the **AEGIS** ensemble under **Hard Voting** regime, as presented in Eq. 6. For a benign input x , let X_l be the binary random variable representing the decision of the l -th layer’s detector:

$$X_l = \phi_{hard}(S_l(x)) = \mathbb{I}(S_l(x) > b_l) \quad (9)$$

where $X_l = 1$ indicates a False Positive (FP) at layer l , and $X_l = 0$ indicates a correct rejection.

Assumption 1 (Bounded Single-Layer Error).

Due to the distribution-aware calibration described in Sec. 3, the FPR of each individual layer is strictly bounded by β :

$$P(X_l = 1) = \mathbb{E}[X_l] \leq \beta \quad (10)$$

Assumption 2 (Approximate Independence).

We model the layer-wise decisions $\{X_l\}_{l=1}^L$ as independent Bernoulli random variables. While deeper layers in LLMs exhibit functional dependencies, non-adjacent layers often capture distinct semantic features (Zhang et al., 2024). This independence assumption provides a theoretical upper bound for the worst-case analysis.

We aim to bound the Ensemble FPR, Q_{ens} , defined as the probability that the sum of votes exceeds the consensus quorum Γ :

$$Q_{ens} = P\left(\sum_{l=1}^L X_l \geq \Gamma\right) \quad (11)$$

Let $S_L = \sum_{l=1}^L X_l$ be the total number of positive votes. Under the independence assumption, S_L follows a sum of independent bounded random variables. The number of false positives is:

$$\mathbb{E}[S_L] = \sum_{l=1}^L \mathbb{E}[X_l] \leq L\beta \quad (12)$$

We apply **Hoeffding’s Inequality** (Hoeffding, 1963), which provides an upper bound on the probability that the sum of bounded independent random variables deviates from its expected value by more than a certain amount t .

Lemma 1 (Hoeffding’s Inequality). *Let $X_1, \dots, X_n$ be independent random variables such that $a_i \leq X_i \leq b_i$. Let $S_n = \sum X_i$. Then for any $t > 0$:*

$$P(S_n - \mathbb{E}[S_n] \geq t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right) \quad (13)$$

In our specific case: (1) The variables are binary, so $a_i = 0, b_i = 1$, and $(b_i - a_i)^2 = 1$; (2) The denominator becomes $\sum_{i=1}^L 1 = L$; (3) We are interested in the probability of the sum exceeding Γ . We rewrite the event $S_L \geq \Gamma$ in terms of deviation from the expectation $L\beta$.

Let $t = \Gamma - L\beta$. We assume the voting threshold is set higher than the expected single-layer error rate (i.e., $\Gamma > L\beta$), ensuring $t > 0$. We substitute these into Hoeffding’s bound:

$$\begin{aligned} Q_{ens} &= P(S_L \geq \Gamma) \\ &= P(S_L - L\beta \geq \Gamma - L\beta) \\ &\leq P(S_L - \mathbb{E}[S_L] \geq \Gamma - L\beta) \end{aligned} \quad (14)$$

Applying the Lemma with $t = \Gamma - L\beta$:

$$Q_{ens} \leq \exp\left(-\frac{2(\Gamma - L\beta)^2}{L}\right) \quad (15)$$

To provide a more intuitive form related to the *voting ratio* versus the *error rate*, we factor out L from the numerator term $(\Gamma - L\beta)^2$:

$$\begin{aligned} (\Gamma - L\beta)^2 &= \left(L\left(\frac{\Gamma}{L} - \beta\right)\right)^2 \\ &= L^2\left(\frac{\Gamma}{L} - \beta\right)^2 \end{aligned} \quad (16)$$

Substituting this back into the exponent:

$$\begin{aligned} Q_{ens} &\leq \exp\left(-\frac{2 \cdot L^2 \left(\frac{\Gamma}{L} - \beta\right)^2}{L}\right) \\ &= \exp\left(-2L\left(\frac{\Gamma}{L} - \beta\right)^2\right) \end{aligned} \quad (17)$$

Noting that $L = |\mathcal{L}_{vote}|$, we arrive at the final bound presented in the main text:

$$Q_{ens} \leq \exp\left(-2|\mathcal{L}_{vote}|\left(\frac{\Gamma}{|\mathcal{L}_{vote}|} - \beta\right)^2\right) \quad (18)$$

Remark on Convergence. This result demonstrates that the ensemble error rate decays exponentially with the number of layers $|\mathcal{L}_{vote}|$. The decay rate is controlled by the square of the **Safety Margin** $(\frac{\Gamma}{L} - \beta)$. As long as the consensus requirement (e.g., $\Gamma/L = 0.5$) is distinct from the single-layer noise floor ($\beta \approx 0.1$), the probability of a false alarm vanishes as more layers are added. ■

A.2 Stability of Soft Voting

In the **Soft Voting** regime (ϕ_{soft}), **AEGIS** operates as a signal accumulator. Consider a stealthy attack that induces a weak activation μ_{adv} at each layer, submerged in background noise σ_{noise} . While the single-layer Signal-to-Noise Ratio (SNR) is μ_{adv}/σ_{noise} , the ensemble sums the consistent semantic signal linearly ($\propto |\mathcal{L}|$) while the independent noise components sum orthogonally ($\propto \sqrt{|\mathcal{L}|}$). The ensemble SNR scales by $\sqrt{|\mathcal{L}|}$, empowering **AEGIS** to detect micro-perturbations that are statistically indistinguishable from noise in any single projection but conspicuous in the aggregate.

A.3 Poof of Ensemble Robustness Radius

Proposition 2 (Ensemble Latent Robustness Radius). *Let $\mathcal{R}(x) = \{R_1(x), R_2(x), \dots, R_{|\mathcal{L}|}(x)\}$ be the set of exact latent radii for all monitored layers calculated via Eq. (7). Let $R_{(\Gamma)}$ denote the Γ -th smallest value in $\mathcal{R}(x)$ (i.e., the Γ -th order statistic). The robustness radius of the **AEGIS** ensemble, $R_{ens}(x)$, is determined by:*

$$R_{ens}(x) = R_{(\Gamma)} \quad (19)$$

Any latent perturbation δ with $\|\delta\|_2 < R_{(\Gamma)}$ is mathematically guaranteed to preserve the safe classification of the ensemble.

Proof. Consider an arbitrary perturbation δ with norm $\|\delta\|_2 < R_{(\Gamma)}$. By the definition of order statistics, there exist at most $\Gamma - 1$ layers where the layer-specific safety margin $R_l(x)$ is smaller than or equal to $\|\delta\|_2$. For all other layers (at least $|\mathcal{L}| - \Gamma + 1$ layers), the perturbation is insufficient to cross the decision boundary:

$$\begin{aligned} \langle h^{(l)} + \delta, w_l^* \rangle &\leq \langle h^{(l)}, w_l^* \rangle + \|\delta\|_2 \\ &< S_l(x) + R_l(x) = b_l \end{aligned} \quad (20)$$

Consequently, the number of positive votes is strictly less than the quorum Γ . The ensemble decision function $\mathbb{I}(\sum v_l \geq \Gamma)$ evaluates to 0, ensuring the input remains classified as safe. □

A.4 Details of AEGIS

We conduct our experiments using PyTorch 2.4.0. All experiments are performed on a single NVIDIA A100-8G GPU. For the default setting of Qwen3-4B in the main experiments, we select the start position as **0.5** (representing the relative network depth, e.g., layer 16 for a 32-layer model) and employ a voting window of $|\mathcal{L}_{vote}| = 3$ layers. For

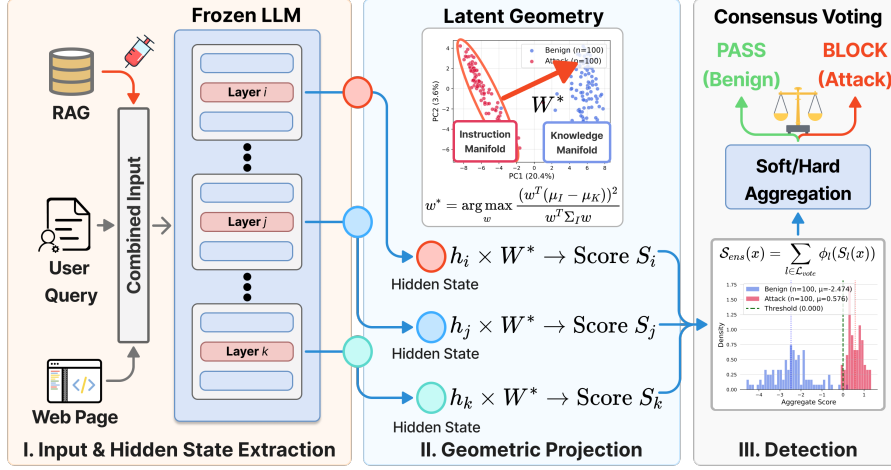


Figure 9: Conceptual illustration of the AEGIS.

Llama3-8B and Llama3.1-8B, we select the start position as **0.5** and employ a voting window of $|\mathcal{L}_{vote}| = 7$ layers. By default, we utilize the **Soft Voting** mechanism as the default aggregation strategy. The covariance matrices Σ_I and projectors are estimated using a computationally efficient calibration set consisting of only 200 benign samples and 200 malicious samples randomly sampled from the training corpus. The threshold to limit the impact on utility strictly bration was selected by 0.05 FPR on the hold-out benign set (training set). The selected threshold was applied to all test datasets and attacks all the time. Note that there are no sample overlaps between the training corpus and the test set. The evaluations for both **AEGIS** and all baselines were run 5 times with different random seeds, and we report the mean value of the final results.

A.5 Implementation of Baselines

We compare **AEGIS** against a comprehensive suite of industrial guardrails and research-based detection methods. All baselines are implemented following their official repositories or standard configurations described in their respective papers. For all methods, thresholds are tuned on the same validation benign set to achieve target FPR (1% unless otherwise stated). Test FPR/FNR are reported on a disjoint test set.

External Classifiers.

- **PIBert** (BlueteamAI, 2024): We utilize the fmops/distilbert-prompt-injection checkpoint, a DistilBERT model fine-tuned specifically for detecting prompt injection attacks.

- **DeBERTa** (ProtectAI.com, 2023): We employ the deberta-v3-base-prompt-injection model, which is optimized for identifying heuristic injection patterns.
- **PromptGuard (PrGuard)** (Meta, 2024): We use meta-llama/Prompt-Guard-86M, a lightweight classifier designed to protect LLM-based applications from malicious inputs.

For all external classifiers, we apply their original (default) classification threshold. Note that this is also the optimal setting given by the official recommendation.

Perplexity-based Detectors. Following Alon et al. (Alon and Kamfonas, 2023), we implement two variants based on the hypothesis that adversarial inputs exhibit anomalous perplexity:

- **PPL¹**: Uses GPT-2-Small as the reference model to calculate windowed perplexity.
- **PPL²**: Uses Vicuna-7B as a more capable reference model.

Internal & LLM-based Defenses.

- **InjecGuard** (Li and Liu, 2024): We leverage the official opensourced checkpoint and code for implementation.
- **Attention Tracker (AttTracker)** (Hung et al., 2025): We implement the attention monitoring mechanism that flags inputs when the attention weights assigned to the external context exceed a specified threshold, indicating a “hijacking” of the model’s focus.

- **LLM Naive** ([Armstrong, 2023](#)): We employ a “defense-via-prompting” approach where a separate LLM (Vicuna-7B-V1.3) serves as a judge. The model is prompted with a system instruction to classify the user input.
- **Known-Answer (KA)** ([Nakajima, 2022](#)): We verify the integrity of the system prompt by appending a secret “canary” token sequence. If the model’s output fails to reproduce the canary correctly (indicating the instruction was overridden), the input is flagged as an attack.
- **PIShield** ([Zou et al., 2025](#)): We implement this defense by extracting the internal representation of the final prompt token from a specific “injection-critical layer” (middle layer in this paper). A simple linear classifier (MLP) is trained on these latent features to distinguish between clean and contaminated prompts based on their intrinsic spectral differences. Note that for fair comparison, we do not include all the training data (20000 samples) and align with **AEGIS** using 200 samples. This might be why PIShield starkly contradicts the original results. Note that we also have to admit that PIShield also exhibits good performance (close but lower than **AEGIS**) when full datasets are leveraged for training, but this validate the data efficiency of **AEGIS** over PIShield as well.

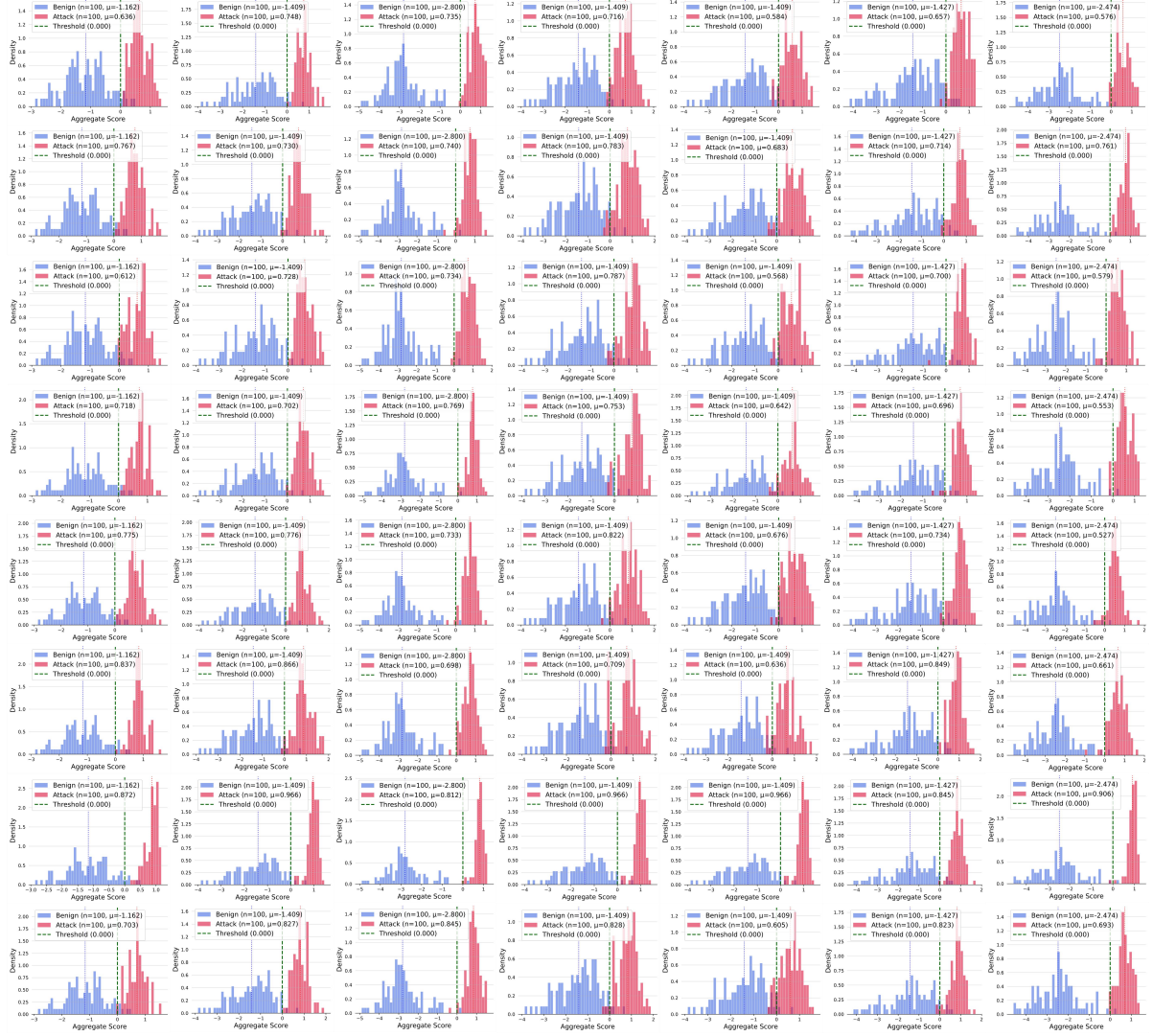


Figure 10: Distribution of the soft voting score by AEGIS. From top to bottom, the IPI attacks are: Combined Attack, Escape Character, Fake Completion, Context Ignoring, Naive Attack, NeuralExec, Pleak and Universal Injection. From left to right, the corresponding datasets are: gigaword, hsol, jfleg, mrpc, rte, sms-spam and sst2

Attack ↓	Model →		Llama3-8B									Llama3.1-8B								
	Start Ratio →	Num Layers →	0.5			0.67			0.6			0.5			0.67			0.6		
			3	5	7	3	5	7	3	5	7	3	5	7	3	5	7	3	5	7
Combined Attack	ACC ↑		0.9712	0.9736	0.9761	0.9713	0.9640	0.9739	0.9705	0.9647	0.9604	0.8823	0.9689	0.9704	0.9564	0.9521	0.9549	0.9727	0.9684	0.9609
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0000	0.0004	0.0004	0.0034	0.0020	0.0021	0.0039	0.0026	0.0011	0.2234	0.0443	0.0391	0.0611	0.0657	0.0563	0.0306	0.0411	0.0503
Escape Character	ACC ↑		0.9707	0.9727	0.9752	0.9696	0.9623	0.9725	0.9700	0.9639	0.9589	0.9473	0.9800	0.9746	0.9633	0.9601	0.9624	0.9724	0.9697	0.9659
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0011	0.0021	0.0021	0.0069	0.0054	0.0050	0.0050	0.0043	0.0043	0.0934	0.0220	0.0309	0.0474	0.0497	0.0411	0.0311	0.0386	0.0403
Fake Completion	ACC ↑		0.9709	0.9732	0.9759	0.9701	0.9641	0.9739	0.9713	0.9647	0.9606	0.9196	0.9714	0.9706	0.9583	0.9544	0.9576	0.9710	0.9656	0.9606
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0007	0.0011	0.0007	0.0057	0.0017	0.0021	0.0025	0.0026	0.0009	0.1489	0.0391	0.0389	0.0574	0.0611	0.0509	0.0340	0.0469	0.0509
Context Ignore	ACC ↑		0.9707	0.9727	0.9752	0.9709	0.9641	0.9734	0.9711	0.9647	0.9600	0.9229	0.9753	0.9707	0.9596	0.9571	0.9604	0.9723	0.9683	0.9621
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0011	0.0021	0.0021	0.0043	0.0017	0.0032	0.0029	0.0026	0.0020	0.1423	0.0314	0.0386	0.0549	0.0557	0.0451	0.0314	0.0414	0.0477
Naive Attack	ACC ↑		0.9705	0.9727	0.9752	0.9691	0.9624	0.9723	0.9704	0.9636	0.9590	0.9487	0.9801	0.9754	0.9657	0.9631	0.9644	0.9741	0.9726	0.9680
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0014	0.0021	0.0021	0.0077	0.0051	0.0054	0.0043	0.0049	0.0040	0.0906	0.0217	0.0291	0.0426	0.0437	0.0371	0.0277	0.0329	0.0360
NeuralExec	ACC ↑		0.9705	0.9721	0.9746	0.9676	0.9606	0.9696	0.9679	0.9619	0.9584	0.9047	0.9731	0.9731	0.9653	0.9609	0.9629	0.9721	0.9731	0.9646
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0014	0.0032	0.0032	0.0109	0.0089	0.0107	0.0093	0.0083	0.0051	0.1786	0.0357	0.0337	0.0434	0.0483	0.0403	0.0317	0.0317	0.0429
PLeak	ACC ↑		0.9700	0.9725	0.9737	0.9610	0.9550	0.9638	0.9700	0.9560	0.9520	0.9550	0.9890	0.9880	0.9850	0.9840	0.9820	0.9860	0.9870	0.9840
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0025	0.0025	0.0050	0.0240	0.0200	0.0225	0.0050	0.0200	0.0180	0.0780	0.0040	0.0040	0.0040	0.0020	0.0020	0.0040	0.0040	0.0040
Universal Injection	ACC ↑		0.9712	0.9734	0.9761	0.9666	0.9601	0.9707	0.9707	0.9627	0.9577	0.9119	0.9793	0.9769	0.9681	0.9599	0.9596	0.9747	0.9734	0.9659
	FPR ↓		0.0575	0.0525	0.0475	0.0540	0.0700	0.0500	0.0550	0.0680	0.0780	0.0120	0.0180	0.0200	0.0260	0.0300	0.0340	0.0240	0.0220	0.0280
	FNR ↓		0.0000	0.0007	0.0004	0.0129	0.0097	0.0086	0.0036	0.0066	0.0066	0.1643	0.0234	0.0263	0.0377	0.0503	0.0469	0.0266	0.0311	0.0403

Table 2: Detection performance on Llama3-8B and Llama3.1-8B regarding different number of layers and position.